



KRN-RPR-001

Kernellum Reproducibility Guide

Clean-room instructions for reproducing the public compiler, verification, and synthesis evidence.

VERSION	v0.2	DATE	September 2026
AUTHOR	Sushanth Dasari	LAB	Kernellum Research

This guide is intended for a new machine or independent reproducer. It describes the expected commands, artifacts, checkpoints, and failure boundaries for Kernellum Compiler v0.1 and the v0.2 alpha ONNX path.

It does not require a physical FPGA. Physical place-and-route and board measurements are handled separately in the FPGA Bring-up Protocol.

AI-native systems for efficient computing.



01 / PREREQUISITES

Environment

COMPONENT	REQUIREMENT	PURPOSE
Python	3.11 recommended	Compiler, training, quantization, tests
pip	current	Editable package install
Icarus Verilog	system package	Generated RTL simulation
Yosys	system package	Generic + ECP5 family synthesis
ONNX extras	pip extra	v0.2 alpha front-end path

02 / SETUP

Clone and install

```
git clone https://github.com/sushxnthd/kernellum.git cd kernellum python -m pip install -e
'.[test,onnx]'
```

03 / TESTS

Run Python verification

```
pytest -q
```

Expected result: the current public test suite passes. If a test fails, stop and record the commit SHA, Python version, dependency versions, and failing traceback before proceeding.

04 / V0.1

Regenerate v0.1 artifacts

```
python -m kernellum --out artifacts/digits_int8
```

CHECKPOINT	EXPECTED PUBLIC VALUE	WHERE RECORDED
Held-out samples	450	manifest.json
Float accuracy	96.22%	manifest.json
INT8 accuracy	96.44%	manifest.json
Float/INT8 agreement	99.78%	manifest.json
Selected lanes	4	manifest.json
Core cycles	680	manifest.json
Modeled latency	6.8 us @ assumed 100 MHz	manifest.json

05 / EDA

Run RTL simulation and generic synthesis

```
bash scripts/run_eda.sh
```

The generated RTL should pass 32/32 golden-vector cases and Yosys should finish synthesis with 0 CHECK problems. Treat tool-version differences as reproducibility data, not something to hide.



06 / FAMILY SYNTHESIS

Run ECP5 family mapping

```
bash scripts/run_ecp5.sh
```

The current public family-mapped synthesis reports 7,977 LUT4 and 8 MULT18X18D primitives. This step does not perform board-specific place-and-route.

07 / ONNX

Run the v0.2 alpha ONNX path

```
python scripts/build_onnx_demo.py bash scripts/run_onnx_eda.sh
```

The front-end intentionally supports a narrow sequential Gemm/ReLU subset. Unsupported graphs should be rejected explicitly rather than silently accepted.

08 / RECORD

Independent reproduction checklist

- 01 Record the exact Git commit SHA.
- 02 Record Python, Icarus Verilog, and Yosys versions.
- 03 Save pytest output and EDA logs.
- 04 Compare manifest fields to the public evidence snapshot.
- 05 Do not reinterpret the modeled 100 MHz latency as achieved hardware timing.
- 06 If results differ, preserve the differing logs before changing dependencies or code.

09 / NEXT THRESHOLD

Physical continuation

For board-specific place-and-route and measurements, follow the separate Kernellum FPGA Bring-up Protocol and GitHub Issue #1. The board/package/clock must be real and explicitly recorded.